

Day 4: Lab Exercises — Distributed Python Computing

Duration: 4 hours (self-paced hands-on practice)

Topics Covered: SCOOP, Pyro4, RPyC, Celery

Note: These exercises are for personal practice to reinforce distributed programming concepts. Start from Exercise 1 and work sequentially.

Exercise 1: Distributed Scientific Computing with SCOOP

Topic: Scientific task distribution, ZeroMQ transport, `scoop.futures.map()`

Problem Statement

Write a distributed scientific computing program `ex01_scoop_monte_carlo.py` that computes numerical integration or Monte Carlo π estimation across multiple worker processes using SCOOP.

1. The Computational Task:

- Define a worker function `evaluate_subrange(params)`:

```
def evaluate_subrange(subrange_id, samples_count):
    # Perform heavy mathematical simulation for samples_count
    iterations
    # Example: Monte Carlo estimation of points under curve y =
    sin(x) * exp(-x)
    ...
    return {"subrange_id": subrange_id, "hits": hits, "total":
    samples_count}
```

- Each task should simulate a computationally demanding workload using randomized or grid evaluations.

2. Distributed Mapping with SCOOP:

- In the `main()` function, split a total workload of 10,000,000 evaluations into 16 discrete task chunks.
- Distribute the chunks across all available CPU cores/workers using `scoop.futures.map()`:

```
from scoop import futures

results = list(futures.map(worker_function, task_workloads))
```

- Aggregate the partial results from all workers and compute the final integrated estimate.

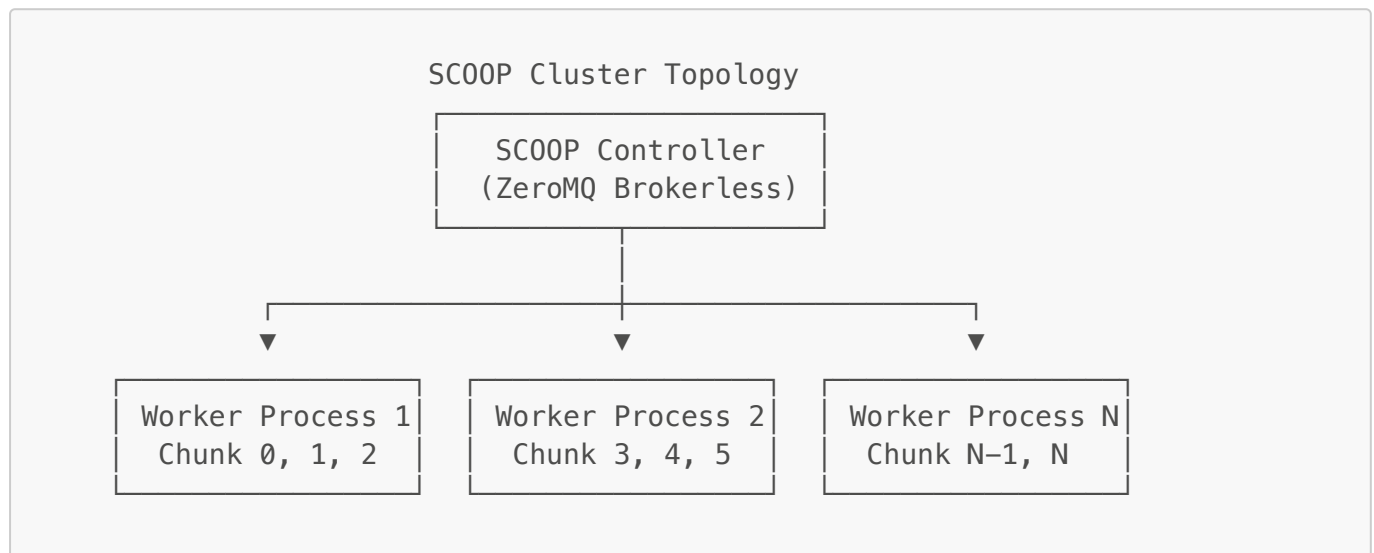
3. Running and Testing:

- Execute the program using the SCOOP runner:

```
python3 -m scoop ex01_scoop_monte_carlo.py
```

- Measure and print the elapsed execution time. Compare this against running the same list of tasks sequentially in a standard `for` loop.

Architecture Overview



Hints & Guidelines

- Ensure worker functions are defined at the **module level** (not inside `main()`).
- Use `if __name__ == "__main__":` to guard your launcher code.
- If testing on multiple machines via SSH, you can pass `--hostfile hosts.txt` to the SCOOP runner.

Exercise 2: Remote Method Invocation with Pyro4 (CLI Menu Application)

Topic: Remote Method Invocation (RMI), `@Pyro4.expose`, Pyro Daemon, Client Proxy, Interactive CLI Menu

Problem Statement

Build a distributed client-server application implementing a **Remote Banking & Wallet Service** using Pyro4. The client must interact with the remote object through an **interactive, menu-driven CLI interface**.

Part A: The Remote Server (`banking_server.py`)

1. Define a class `BankAccountService` decorated with `@Pyro4.expose`:

```

import Pyro4

@Pyro4.expose
class BankAccountService:
    def __init__(self):
        self.balance = 1000.0
        self.transactions = [] # List of strings describing
        transactions

    def get_balance(self):
        ...

    def deposit(self, amount):
        # Validate amount > 0, update balance, record in transaction
log
        ...

    def withdraw(self, amount):
        # Validate amount > 0 and balance >= amount, deduct, record
log
        # Return error string if insufficient funds
        ...

    def get_statement(self):
        # Return copy of transactions list
        ...

```

2. In the server driver:

- Create a `Pyro4.Daemon()`.
- Register the `BankAccountService` class with the daemon.
- Print the generated Pyro URI (`PYRO:obj_xxx@localhost:port`).
- Start the request loop with `daemon.requestLoop()`.

Part B: The Interactive Client (`banking_client.py`)

1. Connect to the remote server using the URI:

```

uri = input("Enter Bank Service URI (or paste from server): ").strip()
bank = Pyro4.Proxy(uri)

```

2. Implement a **menu-driven CLI loop** providing the following options:

```

=====
PYRO4 REMOTE BANKING SERVICE (CLI)
=====
1. Check Current Balance
2. Deposit Money

```

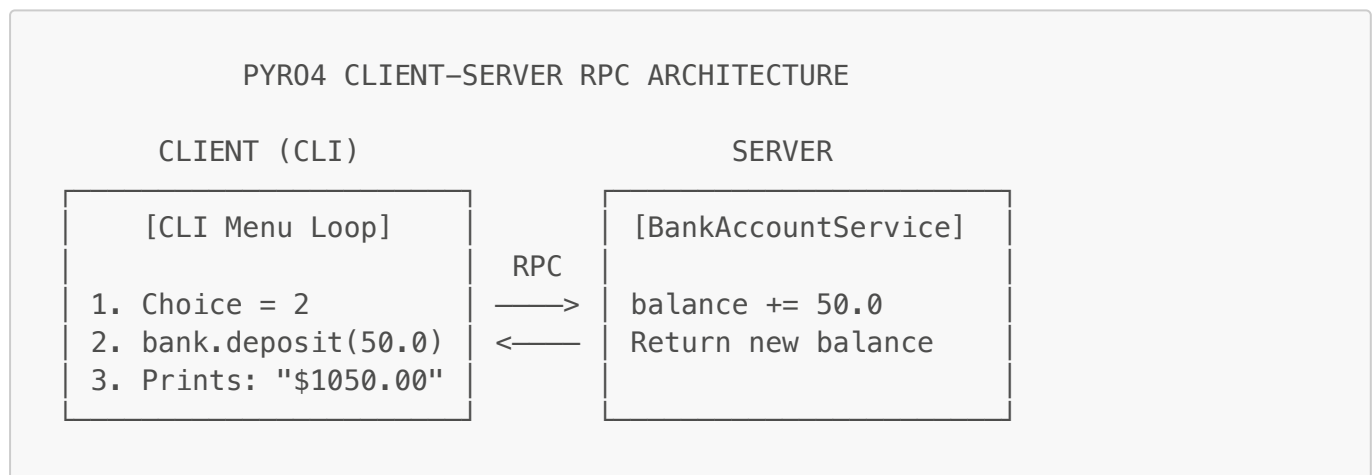
3. Withdraw Money
4. View Transaction Statement
5. Exit Application

Enter your choice [1-5]:

3. Behavior Requirements:

- **Option 1:** Calls `bank.get_balance()` remotely and displays formatted currency (e.g., `Current Balance: $1,250.00`).
- **Option 2:** Prompts user for deposit amount, calls `bank.deposit(amt)` remotely, and prints confirmation with new balance.
- **Option 3:** Prompts user for withdrawal amount, calls `bank.withdraw(amt)` remotely, and handles success or insufficient fund messages returned by the server.
- **Option 4:** Calls `bank.get_statement()` remotely and displays a numbered list of all past transactions.
- **Option 5:** Prints a goodbye message and breaks the loop.
- Catch `ValueError` on bad numeric inputs and `Pyro4.errors.CommunicationError` if the remote server goes down.

Architecture Overview



Exercise 3: Transparent RPC with RPyC (CLI Menu Application)

Topic: Remote Procedure Call (RPC), `rpyc.Service`, `exposed_*` methods, `ThreadedServer`, Interactive CLI Menu

Problem Statement

Build a distributed client-server application implementing a **Remote System Inspector & Math Utility Service** using RPyC. The client must interact with the remote server through an **interactive, menu-driven CLI interface**.

Part A: The Remote Server (`inspector_server.py`)

1. Define a service class inheriting from `rpyc.Service`:

```

import rpyc
from rpyc.utils.server import ThreadedServer
import os
import platform
import time

class SystemInspectorService(rpyc.Service):
    def exposed_get_system_info(self):
        """Returns a dictionary containing server OS, hostname, and
        CPU count."""
        return {
            "system": platform.system(),
            "release": platform.release(),
            "hostname": platform.node(),
            "cpu_count": os.cpu_count(),
            "server_time": time.strftime("%Y-%m-%d %H:%M:%S")
        }

    def exposed_list_files(self, directory_path="."):
        """Returns a list of filenames in the specified directory on
        the server."""
        ...

    def exposed_compute_powers(self, base, max_exponent):
        """Returns a dictionary mapping exponent to base^exp computed
        on the server."""
        ...

    def exposed_execute_expression(self, expression_str):
        """Safely evaluates a basic mathematical expression on the
        server."""
        ...

```

2. Start the service with `ThreadedServer` on a chosen port (e.g. port `18861`):

```

if __name__ == '__main__':
    server = ThreadedServer(SystemInspectorService, port=18861)
    print("RPyC System Inspector Server running on port 18861...")
    server.start()

```

Part B: The Interactive Client (`inspector_client.py`)

1. Connect to the RPyC server:

```

import rpyc

```

```
conn = rpyc.connect("localhost", 18861)
service = conn.root
```

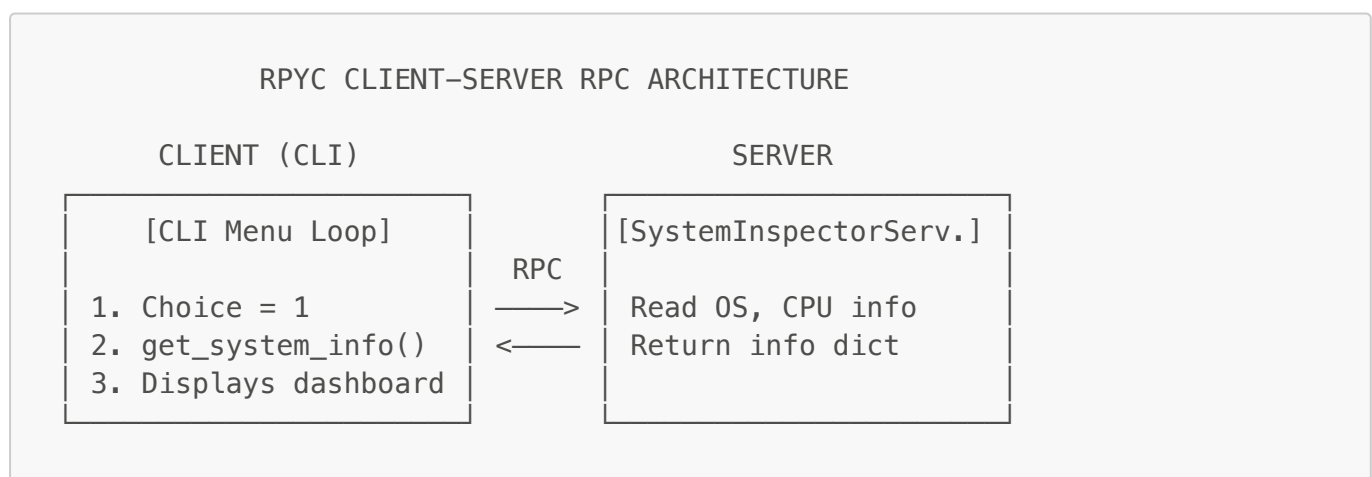
2. Implement a **menu-driven CLI loop** with the following options:

```
=====
      RPYC REMOTE SYSTEM INSPECTOR (CLIENT CLI)
=====
1. Inspect Remote Host System Info
2. List Files on Remote Directory
3. Compute Exponential Table on Remote Host
4. Evaluate Math Expression on Remote Host
5. Disconnect & Exit
-----
Enter your choice [1-5]:
```

3. Behavior Requirements:

- **Option 1:** Calls `service.get_system_info()` and prints a formatted dashboard showing the server's OS, hostname, core count, and current server timestamp.
- **Option 2:** Prompts the user for a folder path on the server (defaulting to `.` current folder), calls `service.list_files(path)`, and displays the files found.
- **Option 3:** Prompts user for a base (e.g., 2) and maximum exponent (e.g., 10), calls `service.compute_powers(base, exp)`, and prints the calculated table.
- **Option 4:** Prompts user for a mathematical string (e.g., `"2**32 - 1"` or `"math.sqrt(144)"`), evaluates it on the remote server, and prints the result.
- **Option 5:** Closes connection with `conn.close()` and exits cleanly.

Architecture Overview



Exercise 4: Distributed Task Processing with Celery

Topic: Celery, Message Broker (RabbitMQ / Redis), Worker Pool, `@app.task`, `.delay()`, `AsyncResult`, Workflow Pipelines

Problem Statement

Build an asynchronous background processing pipeline using Celery that simulates an **Order & Invoice Processing System**:

Step 1: Celery Application Configuration (`celery_config.py`)

Configure a Celery application pointing to a local broker (RabbitMQ or Redis):

```
from celery import Celery

app = Celery(
    'order_system',
    broker='redis://localhost:6379/0',    # Or pyamqp://guest@localhost//
    for RabbitMQ
    backend='redis://localhost:6379/1'   # Or rpc:// for RabbitMQ result
    backend
)
```

Step 2: Task Definitions (`tasks.py`)

Define three distinct distributed tasks:

1. `@app.task def validate_order(order_id, items, total_amount):`
 - Validates that `items` is non-empty and `total_amount > 0`.
 - Simulates validation latency with `time.sleep(1)`.
 - Returns `{"order_id": order_id, "status": "VALIDATED", "total": total_amount}`.
 2. `@app.task def calculate_tax_and_discount(order_data, tax_rate=0.18):`
 - Takes output dictionary from `validate_order`.
 - Computes tax (`total * tax_rate`) and net payable.
 - Returns updated dictionary with calculated tax and final total.
 3. `@app.task def generate_invoice_pdf(order_data, customer_email):`
 - Simulates compiling and rendering an invoice PDF (`time.sleep(2)`).
 - Returns a confirmation string: `"Invoice #INV-{order_id} generated and sent to {email}"`.
-

Step 3: Client Driver (`order_client.py`)

1. Independent Task Submissions:

- Submit 3 separate order validations asynchronously using `.delay()`:

```
async_res = validate_order.delay("ORD-101", ["Item A", "Item B"],
250.0)
```

- Print the generated task UUID immediately: `f"Task Dispatched: {async_res.id}"`.
- Monitor the task status: poll `async_res.status` (PENDING -> SUCCESS) while printing progress dots.
- Retrieve and display the result using `async_res.get()`.

2. Sequential Chained Pipeline:

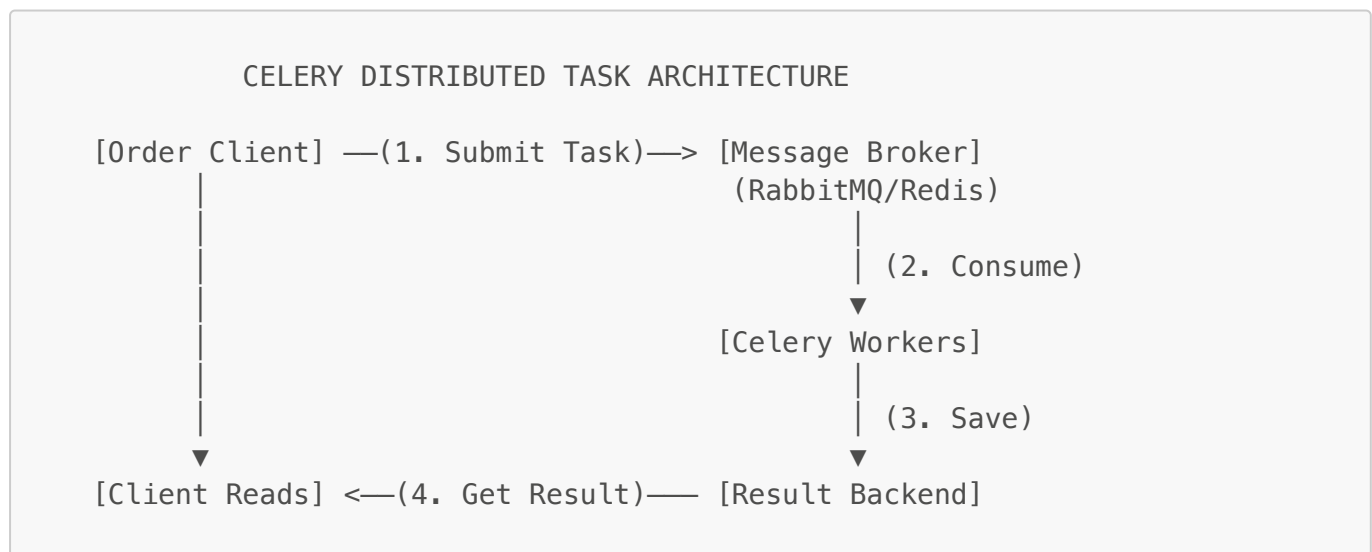
- Construct a Celery **Chain** that links all three tasks sequentially:

```
from celery import chain

order_pipeline = chain(
    validate_order.s("ORD-202", ["Laptop", "Mouse"], 1200.0) |
    calculate_tax_and_discount.s(tax_rate=0.18) |
    generate_invoice_pdf.s(customer_email="customer@example.com")
)

chain_result = order_pipeline.apply_async()
print("Pipeline submitted! Waiting for final invoice...")
final_msg = chain_result.get()
print("Pipeline Output:", final_msg)
```

Architecture Overview



Hints & Guidelines

- Start the Celery worker in a dedicated terminal window:

```
celery -A tasks worker --loglevel=info
```

- If your environment does not have Redis/RabbitMQ running, you can run a local container:

```
docker run -d -p 6379:6379 redis:alpine
```

Or use the simulated worker pattern demonstrated in Demo 01.

Best Practices

- Never pass huge objects (e.g. raw audio/video files) directly through Celery task arguments. Pass lightweight identifiers or file paths.
- Always configure an appropriate Result Backend when using workflows (**chain**, **group**) that require return values.